

REST (RESTful API Execution Engine)

Milenko Burlica

28 July, 2026

Table of Contents

1	 Automatische HTTP-Client-Konfiguration & Ausfallsicherheit.....	4
2	 Engine-spezifische Parameter.....	5
3	 Ordnerstruktur & Service-Mapping (Das Metadaten-Prinzip)	6
3.1	1. Verzeichnis-Struktur.....	6
3.2	2. Die Kern-Dateien eines Kommandos	6
3.3	3. Der Lebenszyklus der Request-Generierung (Runtime-Auflösung).....	7
4	 Unterstützte Operationen (op).....	8
4.1	1. Operation: EXEC (HTTP-Call ausführen).....	8
4.2	2. Operation: ASSERT (Validierung).....	8
4.3	3. Operation: BUFFER (Datenextraktion)	9
5	 Konsole-Logging & Live-Ausgabe (Log Alignment).....	10
6	 Wichtige YAML-Syntaxregeln & Dynamisches Keyword-Handling.....	11
6.1	1. Umgang mit dynamischen Variablen-Scopes	11
6.2	2. Spezifische Header-Keywods: {NULL} und {EMPTY}	11

Die **REST (RESTful API Execution Engine)** dient zur automatisierten Ausführung von HTTP-Anfragen (REST-Calls) sowie zur anschließenden Validierung und Datenextraktion aus den erhaltenen API-Antworten. Sie reagiert dediziert auf den Testschritt-Typ `type: REST`.

 **Das Entkopplungs-Prinzip (Separation of Concerns)** Die REST-Engine trennt die HTTP-Kommunikation strikt von der Datenanalyse:

- Ein HTTP-Aufruf wird einmalig abgesetzt (`op: EXEC`) und das gesamte Antwort-Objekt (Statuscode, Body, Header) unter einem eindeutigen Namen (`response`) im Execution-Context zwischengespeichert.
- Anschließend können beliebig viele Prüfungen (`op: ASSERT`) oder Variablen-Extraktionen (`op: BUFFER`) auf diesem gespeicherten Ergebnis durchgeführt werden, ohne den Server oder API-Endpunkt durch erneute Aufrufe zu belasten.

1 Automatische HTTP-Client-Konfiguration & Ausfallsicherheit

- **SSL Trust-All (Unsafe Client):** Der interne `HttpClient` ist standardmäßig so konfiguriert, dass er alle SSL/TLS-Zertifikate (selbstsignierte Zertifikate im internen Netzwerk) ohne Validierung akzeptiert (`TrustManager` umgeht X509-Prüfungen).
- **Automatischer Retry-Mechanismus:** Bei instabilen Netzwerkverbindungen oder TCP-Verbindungsabbrüchen seitens des Servers (z. B. *Softwaregesteuerte Verbindung / Connection reset* / leere Byte-Antworten im Header-Parser) fängt die Engine den Fehler ab. Sie führt vollautomatisch bis zu **3 Versuche (Retries)** mit einer kurzen Verzögerung von 10ms aus, um Micro-Ausfälle transparent zu kompensieren.
- **Timeout-Schutz:** Jeder Verbindungsaufbau ist durch ein hartes Timeout-Limit von **10 Sekunden** geschützt.

2 Engine-spezifische Parameter

Neben den Standardfeldern besitzt die REST-Engine spezielle Steuerungs-Parameter im YAML:

- **endpoint** (String): Die Basis-URL des Ziel-Servers. Kann hartkodiert oder dynamisch über Umgebungs-Variables geladen werden (z. B. `http://{E[users.serverip]}`).
- **command** (String): Die spezifische API-Route bzw. der vordefinierte Service-Pfad (z. B. `rest.ginosimulator.remove_card`).
- **response** (String, Pflichtfeld): Der eindeutige Schlüssel, unter dem das Antwort-Objekt im Speicher abgelegt (bei `EXEC`) oder referenziert wird (bei `ASSERT` / `BUFFER`).
- **source** (String): Bestimmt den Zielbereich für Prüfungen oder Extraktionen. Erlaubte Werte: `STATUS` , `BODY` oder `HEADERS` .
- **path** (String): Die genaue Position der Daten. Bei `BODY` wird ein standardisierter **JsonPath** (z. B. `$.status.baseNfc.token.value`) verwendet. Bei `HEADERS` wird der exakte Header-Name eingetragen.
- **parameters** (Map, Optional): Dynamische Schlüssel-Wert-Paare, die als Variablen in den Request-Inhalt injiziert werden.

3 Ordnerstruktur & Service-Mapping (Das Metadaten-Prinzip)

Die REST-Engine nutzt das Prinzip "**Convention over Configuration**". Hinter jedem logischen Namen im YAML-Feld `command` verbirgt sich eine feste Verzeichnisstruktur im Framework, die alle technischen Details des HTTP-Requests kapselt. Ohne diese darunterliegenden Metadaten und Templates wäre die Engine nicht in der Lage, einen gültigen REST-Request abzusetzen.

3.1 1. Verzeichnis-Struktur

Der Punkt (`.`) im `command`-Namen wird vom Framework als Pfad-Trennzeichen (`/`) interpretiert. Für das Kommando `rest.ginosimulator.remove_card` sucht das Framework im Ressourcen-Ordner nach folgendem Verzeichnis:

Plaintext

```
src/test/resources/
└─ rest/
    └─ ginosimulator/
        └─ remove_card/
            ├── metadata.json    <-- Technische HTTP-Konfiguration
            └─ request.json      <-- Payload-Template (Request Body)
```

3.2 2. Die Kern-Dateien eines Kommandos

- `metadata.json` (**Technische Definition**): Diese Datei definiert die HTTP-Methode, die relative URL (inklusive des dynamischen Endpunkt-Platzhalters) und die standardmäßig erforderlichen HTTP-Header.

JSON

```
{
  "url" : "{{endpoint}}/simulator/welt-card",
  "method" : "POST",
  "headers" : {
    "Content-Type" : "application/json;charset=UTF-8"
  }
}
```

Der Platzhalter `{{endpoint}}` wird zur Laufzeit automatisch durch den aktuellen Wert des Feldes `endpoint` aus dem YAML-Testschritt ersetzt.

- `request.json` (**Payload-Template**): Diese Datei enthält die JSON-Struktur, die als Body (Nutzdaten) an den Server gesendet wird. Hier werden Platzhalter definiert, die entweder globale Umgebungsvariablen (`E[...]`) oder dynamische Werte aus dem Laufzeit-Buffer (`B[...]`) referenzieren.

JSON

```
{
  "nr" : "{E[users.nr]}",
  "street" : "{B[street]}"
  "land" : "HU"
}
```

3.3 3. Der Lebenszyklus der Request-Generierung (Runtime-Auflösung)

Wenn die Engine `op: EXEC` ausführt, läuft im Hintergrund folgender Prozess ab:

1. **Laden:** Die Engine öffnet den Zielordner basierend auf dem `command` -Namen i liest `metadata.json` und `body.json` ein.
2. **Injektion:** Der Platzhalter `{{endpoint}}` in der URL wird durch die im YAML-Schritt definierte `endpoint` -URL ersetzt.
3. **Parsing:** Die Engine scannt die `body.json` . `{E[... ]}` wird über den `EnvironmentManager` aufgelöst, `{B[... ]}` wird aus dem aktuellen Testlauf-Speicher injiziert. Eventuelle `parameters` aus dem YAML überschreiben oder ergänzen diese Werte final.

4 Unterstützte Operationen (op)

4.1 1. Operation: EXEC (HTTP-Call ausführen)

Erstellt und sendet einen HTTP-Request basierend auf den konfigurierten Parametern, der geladenen `metadata.json` und dem aufgelösten `body.json`.

YAML

```
- type: REST
  op: EXEC
  command: rest.ginosimulator.insert_card
  endpoint: http://{E[users.serverip]}
  response: response_rest2
  parameters:
    street: "first.lady"
```

4.2 2. Operation: ASSERT (Validierung)

Überprüft bestimmte Werte aus dem HTTP-Antwort-Objekt. Jede fehlgeschlagene Assertion bricht den Testlauf sofort ab (Fail-Fast).

Quelle (source)	Aktion (action)	Beschreibung / Verwendung
STATUS	EQUALS , LESS_THAN , GREATER_THAN	Validiert numerisch den HTTP-Statuscode (z. B. <code>expected: 200</code>).
BODY	EQUALS , NOT_EQUALS , CONTAINS	Prüft den via JsonPath extrahierten Textwert.
BODY	IS_EMPTY / IS_NOT_EMPTY	Überprüft, ob ein JSON-Knoten oder der gesamte Body Inhalt besitzt.
BODY	EXISTS	Validiert, ob der JsonPath existiert und Elemente enthält (unterstützt Wildcards <code>[*]</code>).
BODY	COUNT	Zählt Array-Elemente oder Übereinstimmungen. Benötigt optional das Feld <code>value</code> .

Quelle (source)	Aktion (action)	Beschreibung / Verwendung
HEADERS	IS_HEADER_PRESENT	Prüft, ob ein bestimmter Header-Schlüssel in der Antwort existiert.

YAML

```
# Inhaltsprüfung via JsonPath
- type: REST
  op: ASSERT
  response: response_rest3
  source: BODY
  path: $.status.base.registered
  action: EQUALS
  expected: wcard
```

4.3 3. Operation: BUFFER (Datenextraktion)

Extrahiert Daten aus dem `source`-Bereich und speichert sie als globale TestCase-Variable unter dem Attribut `name`. Nachfolgende Testschritte (egal ob REST, SQL, OC) können darauf zugreifen.

Hinweis zu COUNT: Wenn `source: BODY` mit `action: COUNT` kombiniert wird, zählt die Engine die Elemente im Ziel-Array und puffert die Gesamtzahl als String.

YAML

```
# Extrahiert das Token aus dem JSON-Body und speichert es in "${cardtoken}"
- type: REST
  op: BUFFER
  response: response_rest3
  source: BODY
  path: $.status.base.value
  name: wtoken
```

5 Konsole-Logging & Live-Ausgabe (Log Alignment)

Das Logging-System stellt strukturierte Informationen bereit, wenn die Protokollierung aktiviert und im Modul-Kontext auf `isVerbose` gesetzt ist:

Plaintext

```
>>> REST EXEC      : [POST] http://19.18.10.100/api/v1/insert
<<< HEADERS        : Content-Type: application/json;charset=UTF-8
<<< BODY            : {"slot": "base", "nummer": "13"}
>>> RESULT          : SUCCESS (Status: 204)

>>> ASSERT          : [STATUS] EQUALS "204" (Ref: response_rest2)
>>> RESULT          : SUCCESS

>>> REST BUFFER     : name → BODY = $.status.base.value
<<< OUT             : tok_nfc_789456_lts
>>> RESULT          : SUCCESS (Stored)
```

6 Wichtige YAML-Syntaxregeln & Dynamisches Keyword-Handling

6.1 1. Umgang mit dynamischen Variablen-Scopes

Die REST-Engine besitzt ein intelligentes Fallback-System zur Auflösung von Variablen innerhalb von Headern und Request-Bodys. Die Auflösung erfolgt in folgender Priorität:

- `{B[variable_name]}` / `B[variable_name]` : Sucht primär nach dynamischen Werten im Laufzeit-Buffer.
- `{E[env_name]}` / `E[env_name]` : Greift auf Umgebungsparameter zu, die im Framework-Kontext (`tc.getVariables()`) hinterlegt sind.

6.2 2. Spezifische Header-Keywords: `{NULL}` und `{EMPTY}`

Beim Aufbau von HTTP-Headern wertet die Engine zwei spezielle Kontroll-Kywords aus, um Edge-Cases beim API-Testing abzudecken:

- `"{NULL}"` : Der Header wird komplett ignoriert und **nicht** in den HTTP-Request eingebaut (wichtig für Tests von fehlenden Autorisierungs-Headern).
- `"{EMPTY}"` : Der Header wird mit einem leeren String (`" "`) als Wert initialisiert (wichtig für die Übergabe leerer Payload-Typen).

YAML

headers:

```
Authorization: "{NULL}"    # Dieser Header wird nicht gesendet
X-Custom-Trace: "{EMPTY}" # Wird gesendet als: X-Custom-Trace: ""
Content-Type: "application/json"
```